Unix Terminal
Mastery Course

A Complete Course in the Unix/Linux Command Line

Navigation · Permissions · Pipes · Processes · Scripting · Networking · Troubleshooting

10 Chapters · Mini-Labs · Hands-On Exercises · Capstone Project

Chapter 1: Terminal Foundations

What You Will Learn

- Distinguish shell vs. terminal emulator and how they connect to a TTY/PTY.
- Use standard streams (stdin, stdout, stderr) and interpret exit codes.
- Get help for commands with man, --help, and shell built-ins.
- Run and inspect simple commands (echo, printf, history, clearing the screen).

Key Concepts

- **Terminal emulator**: The window/app providing input/output; **shell**: The program reading commands (e.g., bash, zsh).
- **TTY/PTY**: Pseudo-terminals back the emulator; commands read from stdin, write to stdout/stderr.
- **Exit codes**: 0 = success; non-zero = some error/condition; check with `$?`.
- **Built-ins vs. binaries**: Shell built-ins (cd, alias, help) vs. external programs (ls, grep).
- **Standard streams**: stdin, stdout, stderr; redirection uses them (covered deeper in Chapter 4).

Essential Commands

```
echo, printf: Print values; prefer printf for predictable formatting.
```

- clear or Ctrl+L: Clear visible screen.

```
history, !n, !!: Recall commands; use with care when commands contain secrets.
```

```
type, which, command -V: See how a command will resolve (built-in, alias,
path).
```

- man <cmd>, <cmd> --help, help <builtin>: Read docs; / to search, q to quit in man.

Examples

```
# Show how `ls` is resolved

type ls



# Print a formatted line

printf "User: %s\nHome: %s\n" "$USER" "$HOME"
```

```
# View help for a built-in
```

help cd

```
# View manual for grep (press q to exit)
```

man grep

Exercises (Hands-On)

- Identify shell and terminal: Run echo $0 and note your shell; find the terminal emulator name.
- Exit codes: Run true, false, and check $? after each. Create a nonexistent directory listing: ls nosuchdir; note exit code.
- Command resolution: Compare type echo, which echo, and command -V echo. Do the same for cd and ls.
- Use help: Open man printf; find the %d and %s specifiers. Use grep -n "%s" on the man output (with man printf | grep -n "%s" | head).
- History: Run three commands, then inspect history | tail -n 5. Re-run the previous command with !! (ensure it is safe first).

Mini-Lab: Build a Tiny Info Script

Goal: Write a one-liner that prints user, shell, and current TTY.

```
printf "User=%s\nShell=%s\nTTY=%s\n" "$USER" "$SHELL" "$(tty)"
```

Stretch: Save it as ~/bin/whoami-tty (make sure ~/bin is on PATH) and make it executable with chmod +x ~/bin/whoami-tty.

Check Your Understanding

- What is the difference between stdout and stderr, and why does it matter?
- How do you determine whether a command is a shell built-in or an external binary?
- After running a command, how do you check if it succeeded?
- How do you search within a man page?

Common Pitfalls

- Forgetting to quote variables when printing (e.g., paths with spaces) — will be revisited in Chapter 2.
- Assuming which shows aliases/functions; use type or command -V for full picture.
- Using history substitution (!!) blindly; always verify what will re-run.

Prep for Next Chapter

- Practice absolute vs. relative paths with pwd and cd.
- Enable tab completion in your shell if not already on (typically default).

Chapter 2: Navigation and Discovery

What You Will Learn

- Move confidently with absolute and relative paths (., .., ~).
- Use globbing and expansions to target sets of files.
- Apply quoting to control expansion and whitespace handling.
- Discover files and directories quickly with listing and search tools.

Key Concepts

- Absolute vs. relative paths; home ~; current .; parent ..
- Globbing: *, ?, [...] patterns (simple, not regex).
- Brace expansion: {a,b}, {1..3} to generate strings before globbing.
- Quoting: single quotes prevent expansion; double quotes allow variable/command substitution.
- Tab completion: saves keystrokes and reduces typos.

Essential Commands

- pwd: Print current directory.

```
cd <path>, cd -, cd ~user: Change directory (dash returns to previous).
```

```
ls -lah, ls -R: List with details, hidden files, recursion.
```

- tree: Visualize directory structure (if installed).

```
find <path> -name "pattern" -type f: Recursive search by name.
```

- locate <name>: Database-backed search (may require updatedb).
- realpath <path>: Resolve to absolute path.

```
printf %q: Show shell-escaped paths.
```

Examples

pwd

```
cd ~/projects
```

```
ls -lah

cd -              # jump back


# Brace expansion to create paths

mkdir -p demo/{logs,tmp,data}


# Globbing vs. quoting

ls *.txt          # expands

ls "*.txt"        # literal string "*.txt"


# Find files named config.yml under current tree

find . -name "config.yml" -type f


# Locate binaries named python
```

locate bin/python | head

Exercises (Hands-On)

- Make a playground: mkdir -p ~/nav_play/{src,bin,logs} then move around using cd, cd -, cd ...
- Use ls -la to list dotfiles. Add tree ~/nav_play if available.
- Create files: touch ~/nav_play/src/app.{js,py,sh}. Use ls src/*.py and compare to ls "src/*.py".
- Search: From ~/nav_play, run find . -type f -name "*.sh". Try find . -maxdepth 2 -type f.
- Real path: From inside ~/nav_play/src, run realpath ../logs and note the result.

Mini-Lab: Organize a Notes Area

- Create ~/notes/{inbox,archive,ref} and a README.txt inside each.
- Use tree ~/notes (or find ~/notes -maxdepth 2 -type f) to verify.
- List all txt files under notes: find ~/notes -name "*.txt" -print.

Check Your Understanding

- How do glob patterns differ from regular expressions?
- What does cd - do? What about cd ~user?

- Why should you quote paths with spaces or wildcard characters?

Common Pitfalls

- Forgetting quotes when paths contain spaces or */? characters.
- Expecting *.txt to match in subdirectories (it matches only current directory unless used with tools like find or shopt -s globstar in Bash 4+).
- Relying on locate without updating its database; results can be stale.

Prep for Next Chapter

- Keep the ~/nav_play tree for practicing file creation, copying, and permissions.

Chapter 3: File Management and Permissions

What You Will Learn

- Create, copy, move, and delete files/directories safely.
- Inspect file contents with paging and slicing tools.
- Understand Unix permission bits, ownership, and umask defaults.
- Adjust permissions and ownership; use sudo basics.

Key Concepts

- File types: regular files, directories, symlinks.
- Permission triads: user/group/other with rwx; numeric (e.g., 755) vs. symbolic (u+rwx).
- Default permissions and umask: subtracts from defaults to set new file/dir modes.
- Recursive operations: cp -r, rm -r, chmod -R, chown -R (use with care).

Essential Commands

- Create/manage: touch, cp, mv, rm, mkdir -p, rmdir.
- View: cat, less, head, tail, tail -f.
- Permissions: ls -l, chmod, chown, chgrp, umask.
- Elevated: sudo <cmd> (if you have privileges); sudo -l to list allowed commands.

Examples

```
# Copy directory recursively

cp -r src/ backup/src



# Move/rename file

mv notes.txt archive/notes-$(date +%Y%m%d).txt



# Remove a directory tree (confirm path carefully)

rm -r ~/nav_play/tmp



# Inspect permissions and ownership

ls -l ~/nav_play
```

```
# Set permissions: rw-r----- (640)
chmod 640 secrets.txt

# Add execute for user, read for group
chmod u+x,g+r script.sh

# Change owner and group (may require sudo)
sudo chown alice:staff project

# Show current umask
```

umask

Exercises (Hands-On)

- In ~/nav_play, create logs/app.log and data/input.txt. Copy input.txt to archive/input.bak.
- Move logs/app.log to logs/app-1.log, then back using mv.
- Set permissions on archive/input.bak to 640; verify with ls -l.
- Make a directory bin and create a script hello.sh; run chmod 755 bin/hello.sh and execute it with ./bin/hello.sh.
- Check your umask, create a new file, and observe its default permissions.

Mini-Lab: Permission Repair Drill

- Create ~/perm_lab with mkdir -p ~/perm_lab/{private,shared}.
- In private, create secret.txt; set it to 600.
- In shared, create notes.txt; set it to 664 and ensure the directory is 775.
- Verify with find ~/perm_lab -maxdepth 2 -type f -o -type d -exec ls -ld {} +.

Check Your Understanding

- How does umask influence new file permissions?
- When would you prefer symbolic chmod over numeric, and vice versa?
- Why is rm -r risky, and how can you mitigate mistakes?

Common Pitfalls

- Using rm -r or cp -r with an unintended trailing slash or wildcard; double-check paths.
- Forgetting execute bit on scripts, leading to "Permission denied".
- Assuming sudo will preserve your environment; some variables are reset unless using sudo -E (use carefully).

Prep for Next Chapter

- Keep a few sample files ready for experimenting with redirection, pipes, and text processing.

Chapter 4: Streams, Pipes, and Text Tools

What You Will Learn

- Redirect input/output and chain commands with pipelines.
- Use common text-processing tools to filter, transform, and summarize data.
- Understand pipeline exit status and subshell vs. grouping.

Key Concepts

- Standard streams: stdin (0), stdout (1), stderr (2).
- Redirection: > overwrite, >> append, < input, 2> stderr, &> or 2>&1 combine.
- Pipelines: cmd1 | cmd2; each command reads from previous stdout.
- Subshell ( ) vs. grouping { }: environment inheritance and side effects.

```
tee to split output to file and stdout.
```

- Pipeline status: last command's exit code; use set -o pipefail in scripts to catch upstream failures.

Essential Commands and Patterns

```
grep -n "pattern" file (or rg if available) for searching.
```

```
sed 's/old/new/g', sed -n '1,5p' for line filtering.
```

```
awk '{print $1, $3}' for column extraction and light processing.
```

```
cut -d, -f1,3, tr 'A-Z' 'a-z', sort, uniq -c, wc -l.
```

```
xargs -n1 to build commands from input.
```

```
tee file to capture and continue the pipeline.
```

Examples

```
# Save stdout, keep stderr on screen
```

mycmd >out.log

```
# Append both stdout and stderr
```

mycmd &>> combined.log

```
# Simple pipeline: find errors and count

grep -i "error" app.log | wc -l



# Extract column 1 from CSV, count frequency

cut -d, -f1 data.csv | sort | uniq -c | sort -nr | head



# Replace text inline (backup with .bak)

sed -i.bak 's/foo/bar/g' notes.txt



# Use tee to inspect and save

ps aux | grep nginx | tee /tmp/nginx.txt



# Subshell vs. grouping
```

(cd ~/nav_play && ls) # does not change current shell
{ cd ~/nav_play; ls; } # changes current shell

Exercises (Hands-On)

- Redirect: Run echo "hello" > /tmp/hello.txt, then append world with >> and view with cat.
- Stderr: Run ls nosuchfile 2> /tmp/err.log; inspect the error file.
- Pipeline: Count lines containing WARN in a log: grep WARN app.log | wc -l.
- Columns: Given data.csv, extract column 2 and deduplicate: cut -d, -f2 data.csv | sort | uniq.

```
tee: Run ls -l /usr/bin | tee /tmp/bin.txt | head.
```

Mini-Lab: Log Summary One-Liner

Goal: Find the top 5 IPs hitting your web server from access.log.

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -n 5
```

Stretch: Filter only status 500 lines before counting: awk '$9==500 {print $1}' access.log | ....

Check Your Understanding

- How do you redirect only stderr? How do you merge stderr into stdout?
- When do you need tee in a pipeline?
- What is the difference between (cd /tmp; ls) and { cd /tmp; ls; }?

Common Pitfalls

- Overwriting files unintentionally with >; consider set -o noclobber or use >> when appropriate.
- Forgetting quotes around patterns containing ? or * when you intend literal matches.
- Assuming sed -i works the same across systems; BSD sed requires a backup suffix (e.g., -i '').

Prep for Next Chapter

- Keep a sample log and CSV handy; you'll need them while managing processes and automation.

Chapter 5: Processes and Job Control

What You Will Learn

- Inspect running processes and system load.
- Send signals to control or terminate processes.
- Run jobs in the background and manage them interactively.
- Schedule or time-limit commands.

Key Concepts

- PIDs, PPIDs, process groups, and sessions.
- Signals: SIGINT (2), SIGTERM (15), SIGKILL (9), SIGHUP (1).
- Foreground vs. background jobs; job IDs (%1, %2).
- Priorities: nice values and renice to adjust.
- Time-limited execution with timeout.

Essential Commands

- Listing: ps -ef or ps aux, pgrep <name>, top/htop.
- Controlling: kill -SIGTERM <pid>, pkill <pattern>, killall <name> (system-dependent).
- Job control: append &, jobs, fg %1, bg %1, Ctrl+Z to suspend.
- Background resilience: nohup <cmd> &, disown (shell builtin).
- Timing: timeout 30s <cmd>.
- Scheduling: at 10:00 < <file>; crontab -e for recurring tasks (if available).

Examples

```
# Start a long task in background
```

sleep 300 &

```
jobs

fg %1          # bring it back



# Find PIDs for "nginx" and terminate
```

pgrep nginx
pkill -TERM nginx

```
# Run command for at most 5 seconds
```

timeout 5s curl -I https://example.com
top # or htop if installed

```
# Keep running after terminal closes
```

nohup ./server >server.log 2>&1 &

Exercises (Hands-On)

- Start sleep 120 &, list jobs, bring it foreground, then interrupt with Ctrl+C.
- Launch tail -f /etc/hosts and suspend with Ctrl+Z; resume in background with bg %1; bring it back with fg %1.
- Use ps -ef | grep ssh to locate SSH processes; try pgrep ssh.
- Create a CPU-bound loop: python -c "while True: pass" (if python installed); find its PID and terminate with kill -TERM <pid>.
- Run a command with timeout 3s sleep 10 and observe exit status $?.

Mini-Lab: Resource Watchdog

- Start yes > /dev/null & to simulate load (stop it after!).
- In another shell, observe with top or htop.
- Use renice to lower its priority, then kill to stop it.

Check Your Understanding

- What is the difference between SIGTERM and SIGKILL? When would you use each?
- How do you resume a stopped job in the background? In the foreground?
- How do you find all processes matching a name without using ps | grep?

Common Pitfalls

- Killing the wrong process due to an overly broad pattern with pkill.
- Forgetting that nohup still writes output unless redirected.
- Assuming timeout is available on every system; some minimal environments may lack it.

Prep for Next Chapter

- Gather a few repetitive tasks you wish to automate; you'll script them next.

Chapter 6: Shell Scripting Essentials

What You Will Learn

- Write portable shell scripts with safe defaults.
- Use variables, parameter expansion, conditionals, loops, and functions.
- Read user input and work with here-docs/strings.
- Add basic debugging and error handling.

Key Concepts

- Shebang: #!/usr/bin/env bash (or sh); choose shell intentionally.
- Safety flags: set -euo pipefail and IFS=$'\n\t' for stricter behavior.
- Quoting discipline: prefer double quotes around variable expansions.
- Parameter expansion: defaults ${var:-fallback}, substring ${var%/*}, length ${#var}.
- Functions: foo() { ...; } with return codes; local variables (local in bash/zsh).
- Conditionals: [ ], [[ ]] (bash/zsh); test operators -f, -d, -n, -z, integers -eq, strings ==.
- Loops: for, while read -r, until.
- Here-docs cat <<'EOF' ... EOF vs. here-strings cmd <<< "text".

Script Skeleton

```
#!/usr/bin/env bash

set -euo pipefail
```

IFS=$'\n\t'
usage() {

```
  echo "Usage: $0 <input_file>" >&2
```

}
main() {
local file=${1:-}

```
  [[ -z $file ]] && usage && exit 1

  [[ ! -f $file ]] && echo "No such file: $file" >&2 && exit 1

  while IFS=, read -r col1 col2; do
```

```
    printf "%s -> %s\n" "$col1" "$col2"

  done < "$file"
```

}
main "$@"

Exercises (Hands-On)

- Write a script greet.sh that accepts a name and prints "Hello, "; exit with error if no name.
- Create backup.sh that copies files from a source directory to ~/backup with a timestamp; check if source exists.
- Write rename-spaces.sh to replace spaces with underscores in filenames within current directory using a loop.
- Add set -x to one of your scripts to see tracing; remove it afterward.
- Practice reading input safely: while IFS= read -r line; do echo "$line"; done < file.txt.

Mini-Lab: Simple CSV Checker

- Input: CSV file where column 1 must be non-empty and column 3 must be an integer.
- Script tasks:
- Validate args and file readability.
- Read lines with IFS=, read -r c1 c2 c3.
- If c1 empty or c3 not an integer ([[$c3 =~ ^[0-9]+$]]), print an error with line number.
- Exit non-zero if any row fails.

Check Your Understanding

- Why use set -euo pipefail? What pitfalls remain even with it?
- How do <<EOF and <<< differ?
- When should you prefer [[...]] over [...]?

Common Pitfalls

- Forgetting to quote variables, causing word-splitting or globbing surprises.
- Using for f in $(ls) which breaks on spaces; prefer for f in *.txt; do ...; done or find with -print0 and while IFS= read -r -d '' f; do ...; done.
- Relying on bash-specific features in environments where only sh is available.

Prep for Next Chapter

- Collect aliases and prompt tweaks you like; we'll fold them into startup files.

Chapter 7: Environment and Customization

What You Will Learn

- Understand shell startup files and when they run.
- Manage PATH, aliases, and functions for productivity.
- Customize prompts and completions.
- Keep a portable, minimal dotfiles setup.

Key Concepts

- Shell types: login vs. interactive vs. non-interactive; different files sourced.
- Common files: /etc/profile, ~/.profile, ~/.bash_profile, ~/.bashrc, ~/.zshrc.
- PATH order matters; earlier entries win. Prefer appending/prepending intentionally.
- env vs. export: exporting makes variables visible to child processes.
- Aliases for short commands; functions for logic; completions via shell frameworks or bash-completion.
- Prompts: PS1 (primary), PS2 (continuation); use simple, informative prompts.

Essential Snippets

```
# ~/.bashrc (example)

# Safety and PATH

set -o vi                              # or `set -o emacs`

export PATH="$HOME/bin:$PATH"



# Aliases

alias ll='ls -lah'

alias gs='git status'



# Functions
```

mkcd() { mkdir -p "$1" && cd "$1"; }

extract() { case "$1" in *.tar.gz) tar -xzf "$1";; *.zip) unzip "$1";; *) echo "Unknown";; esac; }

```
# Prompt (bash)
```

PS1='\u@\h:\w$ '

Exercises (Hands-On)

- Identify which startup files your shell uses: open a new login shell and echo a marker from each file.
- Add ~/bin to PATH in ~/.profile (login) and verify with echo $PATH after re-login.
- Create an alias lt for ls --tree or ls -R fallback; test it.
- Add a function cproj that cds into your main projects folder.
- Customize your prompt to show user@host:cwd and git branch if available (optional: use __git_ps1).

Mini-Lab: Portable Dotfiles Starter

- Create ~/dotfiles containing bashrc and profile snippets.
- In ~/.bashrc, source the dotfiles version: [-f "$HOME/dotfiles/bashrc"] && . "$HOME/dotfiles/bashrc".
- Add minimal content: PATH update, aliases, mkcd, prompt.
- Test by opening a new terminal; ensure no errors when files are missing on another machine.

Check Your Understanding

- What is the difference between ~/.profile and ~/.bashrc? When is each read?
- How does PATH ordering influence which binary runs?
- When should you choose a function instead of an alias?

Common Pitfalls

- Editing the wrong startup file and wondering why changes do not apply.
- Prepending duplicate PATH entries, leading to long PATH strings and unexpected binaries.
- Overly complex prompts harming performance on slow remote shells.

Prep for Next Chapter

- Generate or locate your SSH keys and think about common remote hosts you access.

Chapter 8: Networking and Remote Workflows

What You Will Learn

- Connect to remote systems securely with SSH.
- Transfer files with scp/sftp and set up SSH config entries.
- Use port forwarding for tunneling.
- Fetch resources and APIs with curl/wget.
- Inspect basic network state with netstat/ss.

Key Concepts

- SSH key pairs: private vs. public; passphrases; agents (ssh-agent / ssh-add).
- ~/.ssh/config for host aliases, identities, ports, and forwarding.
- Port forwarding: local -L and remote -R; SOCKS -D.
- Basic HTTP operations: GET, HEAD; status codes; headers.
- Network sockets and listening ports; privilege differences (<1024 vs higher).

Essential Commands and Snippets

```
# Generate an ed25519 key

ssh-keygen -t ed25519 -C "you@example.com"



# Add key to agent

ssh-add ~/.ssh/id_ed25519



# SSH config entry (~/.ssh/config)
```

Host work
HostName work.example.com
User alice
IdentityFile ~/.ssh/id_ed25519
Port 22

```
# Connect using alias

ssh work
```

```
# Copy files

scp file.txt work:/home/alice/

scp -r dir/ work:/home/alice/dir


# Port forward local 8080 to remote 80

ssh -L 8080:localhost:80 work


# Fetch headers from a site

curl -I https://example.com


# Download a file

wget https://example.com/file.tar.gz


# List listening sockets

ss -tuln | head   # or netstat -tuln
```

Exercises (Hands-On)

- Generate a new SSH key (ed25519). View the public key and copy it to a test host (or a local VM) using ssh-copy-id if available.
- Create an SSH config alias playbox pointing to a host/VM you control; test ssh playbox.
- Transfer a file to the host with scp and verify its checksum on both ends (sha256sum).
- Set up a local port forward: ssh -L 9000:localhost:22 playbox and test ssh -p 9000 localhost in another terminal.
- Use curl -s https://httpbin.org/get | head to inspect JSON; pretty-print with python -m json.tool if jq is unavailable.

Mini-Lab: Remote Log Grabber

- Goal: Fetch /var/log/syslog (or /var/log/messages) from a remote host, store locally with timestamp, and keep permissions sane.
- Steps:
- Create ~/logs locally.

```
scp playbox:/var/log/syslog ~/logs/syslog-$(date +%Y%m%d).log
```

- Run ls -lh ~/logs to verify size and timestamp.

Check Your Understanding

- What is stored in the private key vs. public key? Why protect the private key with a passphrase?
- How does ssh -L differ from ssh -R?
- When would you prefer curl -I over curl -s?

Common Pitfalls

- Incorrect file permissions on ~/.ssh or keys (fix with chmod 700 ~/.ssh and chmod 600 ~/.ssh/*).
- Forgetting to add the correct key to ssh-agent, leading to password prompts.
- Leaving long-running port forwards open and conflicting with local ports.

Prep for Next Chapter

- Save a few failing command examples (PATH issues, permission errors) to practice troubleshooting.

Chapter 9: Troubleshooting and Diagnostics

What You Will Learn

- Diagnose PATH and command resolution issues.
- Fix permission and ownership problems.
- Handle line ending and encoding mismatches.
- Identify disk space and open-file issues.
- Read system logs for clues.

Key Concepts

- PATH lookup order; type -a to see resolution.
- Common errors: "command not found", "permission denied", CRLF line endings, stale sockets, full disks.
- Filesystems and inodes: disk full vs. inode exhaustion.
- Encoding basics: UTF-8 vs. others; file to identify.

Essential Commands

- PATH/debug: echo $PATH, type -a cmd, which -a cmd, hash -r to clear cache (bash).
- Permissions: ls -l, namei -l (if available) to trace permissions along a path.
- Line endings: file, dos2unix, unix2dos.
- Disk: df -h, du -sh *, du -sh .[!.]* for dotfiles, find . -size +100M.
- Open files: lsof -p <pid>, lsof -i :<port>, fuser <file>.
- Logs: dmesg | tail, journalctl -xe (systemd), /var/log/*, tail -f.

Examples

```
# PATH issues

which -a python
```

hash -r # clear hashed commands in bash

```
# Permissions along path
```

namei -l /var/www/html/index.html

```
# Fix CRLF line endings
```

dos2unix script.sh

```
# Disk usage overview

df -h


du -sh ./* | sort -h | tail


# Find large files

find . -type f -size +200M -print


# Port in use
```

lsof -i :8080

Exercises (Hands-On)

- Break and fix PATH: temporarily set PATH=/bin and see which commands fail; restore and run hash -r.
- Create a file with Windows line endings using an editor; detect with file and convert with dos2unix.
- Fill a temp directory with files and find the largest 3: find . -type f -printf '%s %p\n' | sort -nr | head -n 3 (BSD find may need -size +1M -print).
- Start python -m http.server 8000 (if Python exists); check which process holds port 8000 using lsof -i :8000 and stop it.
- Read recent kernel messages: dmesg | tail (may require sudo on some systems).

Mini-Lab: PATH Fixer

- Scenario: mytool exists in /opt/mytool/bin but command not found occurs.
- Steps:
- Verify location: find /opt -maxdepth 2 -type f -name mytool -print.
- Export PATH: export PATH="/opt/mytool/bin:$PATH" in the current shell and test.
- Add the export to your shell startup file responsibly.

Check Your Understanding

- How do you check every directory where a command could be resolved?
- What symptoms indicate CRLF issues in shell scripts?
- How do you differentiate disk-full vs. inode exhaustion?

Common Pitfalls

- Editing PATH with trailing colons or spaces, creating empty entries that resolve to current directory.
- Using sudo without -E and losing environment adjustments needed for a command (use carefully).
- Ignoring error output; stderr often explains the failure—capture it if needed.

Prep for Next Chapter

- Gather a small workflow you can automate end-to-end for the capstone.

Chapter 10: Putting It Together — Capstone

What You Will Learn

- Design and implement a small automation workflow end-to-end.
- Apply navigation, file ops, pipelines, scripting, and remote skills together.
- Add basic ergonomics: usage/help text, logging, and timing.
- Package your work so it is reusable.

Capstone Ideas (Pick One)

- Log summarizer: Parse a log directory, extract top errors, email or save a report.
- Backup sync: Tar or rsync a project to a dated archive or remote host.
- Project scaffolder: Create a directory tree with starter files and git init.
- Remote fetcher: Pull data from an API with curl, transform, and store locally.

Suggested Workflow

- Define inputs/outputs: flags, source paths, destinations, temp directories.
- Draft usage/help: usage() { echo "Usage: $0 [options]"; }.
- Start with a skeleton script: shebang, set -euo pipefail, argument parsing (basic while getopts "o:s:" opt; do ... done).
- Implement core steps:
- Validate paths and permissions.
- Use pipelines for filtering/formatting data.
- Add logging to stderr (e.g., echo "[info] ..." >&2).
- Add safety:
- Trap cleanup: trap 'rm -rf "$TMPDIR"' EXIT for temp work.
- Use mktemp -d for temporary directories.
- Quote all expansions; handle spaces.
- Test incrementally with small datasets; add set -x temporarily when debugging.
- Package: place script in ~/bin, mark executable, document prerequisites.

Example Skeleton (Backup)

```
#!/usr/bin/env bash

set -euo pipefail
```

usage() { echo "Usage: $0 -s <source> -d <dest>" >&2; }

```
while getopts "s:d:" opt; do
```

```
  case $opt in
```

s) src=$OPTARG;;
d) dst=$OPTARG;;
*) usage; exit 1;;

```
  esac
done
```

: "${src:?source required}" "${dst:?dest required}"

```
stamp=$(date +%Y%m%d-%H%M%S)
archive="$dst/backup-$stamp.tar.gz"


mkdir -p "$dst"
tar -czf "$archive" -C "$src" .
echo "Wrote $archive"
```

Exercises (Hands-On)

- Pick one capstone idea and sketch inputs/outputs; write a short README describing purpose and usage.
- Implement argument parsing with getopts; add a -h flag for help.
- Add logging functions (log_info, log_warn) that write to stderr.
- Measure runtime with time ./script.sh ...; experiment with pv in pipelines if available.
- Add a simple test: run the script against sample data and check expected outputs.

Check Your Understanding

- Why is incremental testing crucial when chaining many commands?
- How do you ensure temporary files are always cleaned up?
- What makes a script portable across environments?

Common Pitfalls

- Hardcoding paths and assumptions about the environment (shell, tools installed).
- Skipping error handling around tar, ssh, or curl calls; always check exit codes.

- Forgetting to document prerequisites and usage, making the script hard for others to run.

Next Steps and Further Learning

- Add linting with shellcheck if available.
- Learn make to orchestrate tasks, and deepen git CLI skills.
- Explore container CLIs (Docker/Podman) and advanced shells (zsh/fish) for productivity.